SQL Chatbot for Business Data

Ask questions in plain English. Get SQL, a chart, and a table back.

> **One line:** A web app where a business user types *"Which product sold the most last month?"*, and the app writes safe SQL, runs it on the database, and shows the answer as a chart.

Project type	Full Stack + AI/LLM + Data Analytics
Level	Medium
Build time	About 2 weeks (basic version in 1 week)
Core idea	Text-to-SQL with RAG, safety checks, and auto charts
Good for roles	AI/LLM Engineer, Data Analyst, Analytics Engineer, Full Stack, Backend

1. Problem and Solution

Problem. Business people (sales, HR, ops) have data in databases, but they cannot write SQL. They wait for a data analyst to answer simple questions. This is slow.

Solution. A chatbot that turns English questions into SQL, runs them safely, and shows the result as a chart. No SQL knowledge needed.

Example

User asks	App does	User sees
Which product sold the most last month?	Writes a SELECT with GROUP BY and ORDER BY	Bar chart + table + the SQL
Show monthly revenue for 2025	Writes a SELECT with date grouping	Line chart
Now only for Karnataka	Uses chat memory, adds a WHERE filter	Updated line chart
Delete all orders	Validator blocks it	Message: only read queries are allowed

2. Use Cases

- **Business users:** get answers without waiting for an analyst.
- **Data analysts:** get a first SQL draft fast, then improve it.
- **Startups:** a simple data assistant on top of their own database.
- **Students and learners:** see how English maps to SQL (the app shows the SQL).
- **Upload your own CSV:** chat with any spreadsheet.

3. Features

Core features (build first)

- Plain English to SQL using an LLM.
- Runs the SQL on PostgreSQL and shows a table.
- Auto chart selection (bar, line, pie, number card).
- Shows the SQL used, so the user can trust it.
- Query history.

Smart features (build second)

- **Auto-fix:** if the SQL fails, send the error back to the LLM and retry (up to 2 times).
- **Follow-up questions:** "now only last month" works because the chat has memory.
- **RAG context:** fetch only the needed tables, column meanings, and past good queries.
- **Business glossary:** teach terms like "active customer" and "revenue".
- **Upload CSV:** load it as a table and ask questions.
- **Explain answer:** one or two lines in simple English about the result.

Pro features (build last, for resume impact)

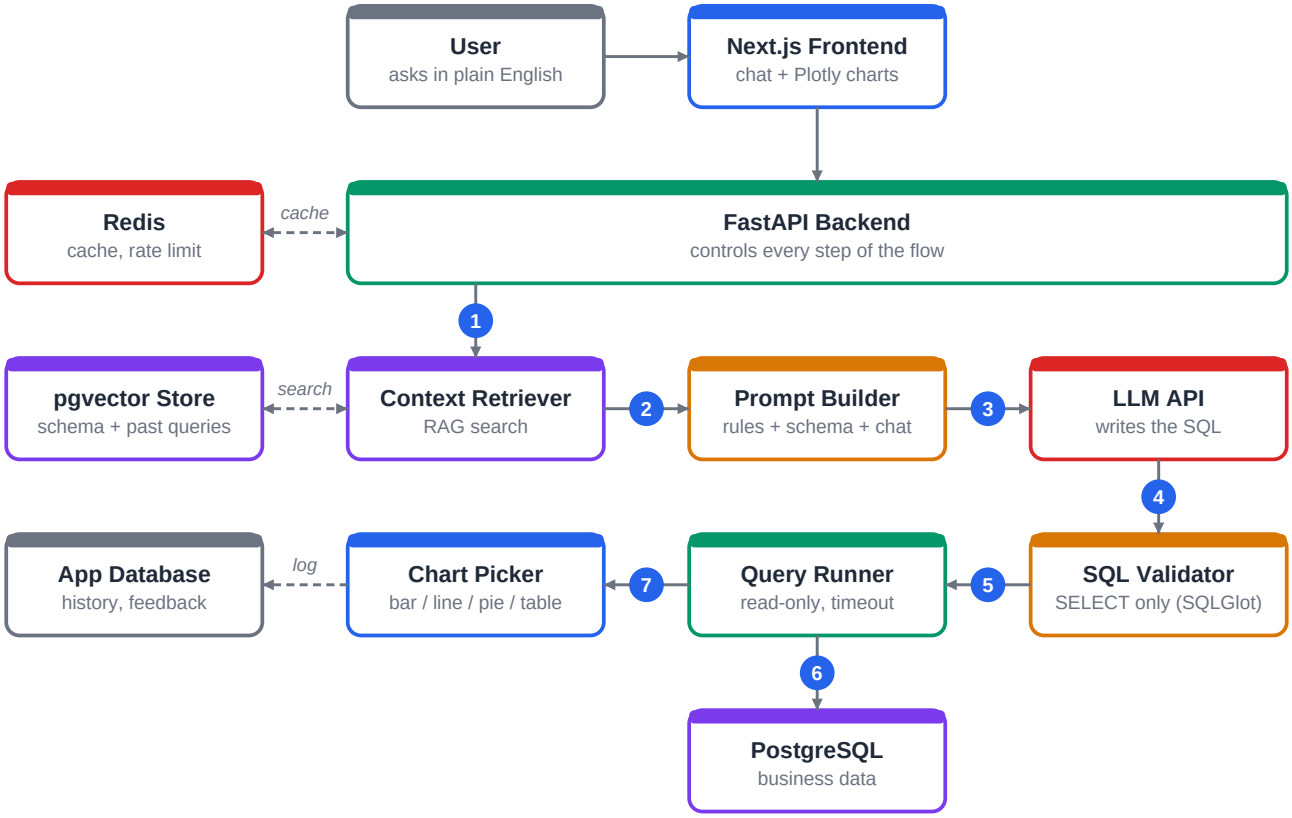
- **Safety layer:** only SELECT, read-only DB user, row limit, query timeout.
- **Evaluation dashboard:** 40 test questions, accuracy score, failed cases.
- **Feedback buttons:** thumbs up or down. Good queries are saved to improve future answers.
- **Multi-database:** PostgreSQL and MySQL support.
- **Cost and speed tracking:** tokens, cost, and time per question.
- **Login and roles:** users only see tables they are allowed to see.

4. Tech Stack

Part	Tools	Why
Frontend	Next.js, Tailwind CSS, Plotly.js	Chat page and interactive charts
Backend	FastAPI (Python)	Fast, simple, great for AI work
LLM	Any LLM API (Claude, OpenAI, Gemini)	Writes the SQL
Business DB	PostgreSQL	Holds the data users ask about
Vector store	pgvector (inside PostgreSQL)	Stores schema info and past queries for RAG
Embeddings	sentence-transformers or an API	Turns text into searchable numbers
SQL safety	SQLGlot	Parses SQL and checks it is safe
Cache	Redis	Caches repeated questions, rate limits
App DB	PostgreSQL (separate schema)	Users, chat history, feedback
Deploy	Docker, Docker Compose, any cloud VM	Easy to run anywhere
Testing	Pytest, custom eval script	Measures SQL accuracy

5. Architecture

The system has four parts: the **frontend**, the **FastAPI backend** that controls the flow, the **AI layer** (retriever, prompt builder, LLM), and the **data layer** (databases and cache).



Numbers 1-7 = order for one question. Dashed lines = supporting storage. Chart, table and SQL then go back to the frontend.

Main components

Component	Job
Next.js Frontend	Chat box, shows chart, table, and SQL. Sends questions to the API.
FastAPI Backend	Main controller. Runs each step in order and handles errors.
Context Retriever	Searches pgvector for the table descriptions, glossary terms, and past queries that match the question.
Prompt Builder	Builds one clear prompt: rules + relevant schema + glossary + examples + chat history + question.
LLM API	Returns SQL only (no extra text).
SQL Validator	Parses SQL with SQLGlot. Allows SELECT only. Blocks DROP, DELETE, UPDATE, multiple statements. Adds LIMIT.
Query Runner	Runs SQL with a read-only DB user and a timeout (for example 10 seconds).
Chart Picker	Chooses a chart from the result shape (see rules below).
pgvector Store	Holds embeddings of table descriptions, column meanings, glossary, and good past queries.
Redis	Caches repeated questions and limits requests per user.

6. Request Flow (One Question)

Step	Name	What happens
1	User asks	User types a question in the chat.
2	Cache check	Backend checks Redis. If the same question was answered recently, return it.
3	Find context (RAG)	Embed the question. Search pgvector for the top 5 to 10 matching table descriptions, glossary terms, and good past queries.
4	Build prompt	Add rules, the matched schema, a few example rows, glossary, chat history, and the question.
5	LLM writes SQL	LLM returns one SELECT query.
6	Validate	SQLGlot checks it. Not SELECT only? Reject. Add LIMIT 1000 if missing.
7	Run	Run on PostgreSQL with the read-only user and a timeout.
8	Auto-fix	If SQL fails, send the error to the LLM and retry up to 2 times.
9	Pick chart	Choose chart type from the result columns.
10	Respond	Send chart data, table, SQL, and a short explanation to the frontend.
11	Save	Store the question, SQL, and result summary in history. Wait for feedback.
12	Learn	If the user clicks thumbs up, save question + SQL as a good example in pgvector.

Chart picker rules

Result shape	Chart
One number	Number card
One date column + one number column	Line chart
One text column + one number column	Bar chart
Few categories (5 or less) showing share	Pie chart
Two number columns	Scatter plot
Anything else	Table only

7. Data Design

Business data (sample)

Use a free dataset such as an e-commerce or superstore sales dataset (Kaggle). Suggested tables:

Table	Main columns
customers	customer_id, name, city, state, signup_date
products	product_id, name, category, price
orders	order_id, customer_id, order_date, status, total_amount
order_items	order_id, product_id, quantity, unit_price

App tables

Table	Purpose
users	Login and roles
chat_sessions, messages	Chat memory and history
query_logs	Question, SQL, time, tokens, cost, success or fail
feedback	Thumbs up or down per answer
schema_docs (pgvector)	Table and column descriptions with embeddings
glossary (pgvector)	Business terms and their meaning
good_queries (pgvector)	Approved question + SQL pairs

8. Safety Plan

Letting an LLM write SQL is risky. Use several layers so one failure does not cause damage.

Layer	What it does
Read-only DB user	The database itself refuses any write, even if the app has a bug.
SQL validator	Only one SELECT statement. Blocks DROP, DELETE, UPDATE, INSERT, ALTER.
Table allow-list	Query can touch only approved tables.
Row limit	Always add LIMIT so huge results do not crash the app.
Timeout	Stop slow queries after about 10 seconds.
Rate limit	Redis limits requests per user.
Prompt injection guard	Treat user text as data. Never let it change the rules.
Audit log	Save every question and SQL that was run.

9. Evaluation (Makes the Project Stand Out)

Most student projects skip this. Doing it shows engineering skill.

- Write **40 test questions** with the correct SQL or the correct answer.
- Mix easy (single table), medium (joins), and hard (grouping, date logic, subqueries).
- Run all questions automatically. Compare the **result** of generated SQL with the correct result.
- Report: **accuracy %**, average time, average cost, and failed cases.
- Show before and after: basic prompt vs RAG + glossary + examples. Example: 62% to 88%.

> **Resume line example:** Built a text-to-SQL chatbot with RAG (pgvector), SQL safety validation, and auto-retry. Reached 88% execution accuracy on a 40-question test set; deployed with Docker.

10. Build Plan

Phase	Time	Tasks
1. Basics	Days 1-3	Load dataset into PostgreSQL. FastAPI endpoint: question to LLM to SQL to result. Simple Next.js chat page.
2. Charts	Days 4-5	Chart picker rules. Plotly charts and table. Show the SQL.
3. Safety	Days 6-7	SQLGlot validator, read-only user, LIMIT, timeout. Auto-fix retry.
4. RAG	Days 8-10	Table and column descriptions in pgvector. Glossary. Retrieve context per question. Chat memory for follow-ups.
5. Quality	Days 11-12	Evaluation script with 40 questions. Feedback buttons and good-query learning. Improve prompts using failed cases.
6. Polish	Days 13-14	CSV upload, login, Redis cache, Docker Compose, deploy, README with screenshots and accuracy numbers.

11. Suggested Folder Structure

```
sql-chatbot/
  backend/
    app/
      main.py            (FastAPI app)
      routes/            (chat, history, upload, feedback)
      services/
        retriever.py     (pgvector search)
        prompt.py        (prompt builder)
        llm.py           (LLM client)
        validator.py     (SQLGlot checks)
        runner.py        (safe query execution)
        charts.py        (chart picker)
      db/                (models, connections)
    eval/
      questions.json     (40 test questions)
      run_eval.py        (accuracy report)
  frontend/
    app/                 (Next.js pages)
    components/          (ChatBox, ChartView, SqlView)
  docker-compose.yml
  README.md
```

12. Challenges and Fixes

Challenge	Fix
LLM writes wrong column names	Send only the relevant schema with column descriptions. Add example rows.
Confusing business terms	Business glossary stored in pgvector.
Wrong date logic ("last month")	Put today's date in the prompt. Add example queries for date ranges.
Dangerous SQL	Validator + read-only user + allow-list.
Big databases (50+ tables)	RAG: send only the top matching tables, not the full schema.
Slow answers	Cache repeated questions in Redis. Use a smaller LLM for simple questions.
High cost	Short prompts, caching, and cost tracking per question.

13. Future Add-ons

- Dashboard builder: pin favorite charts to a page.
- Scheduled reports sent by email.
- Voice questions (speech to text).
- Slack or Telegram bot.
- Anomaly alerts: "sales dropped 30% today".

14. Resume Summary

SQL Chatbot for Business Data | FastAPI, Next.js, PostgreSQL, pgvector, SQLGlot, Plotly, Redis, Docker
- Built a natural-language-to-SQL assistant with RAG over schema docs, a business glossary, and approved past queries.
- Added multi-layer SQL safety (SELECT-only validation, read-only role, row limit, timeout) and auto-retry on errors.
- Created an evaluation harness (40 questions) and improved accuracy from 62% to 88% (replace with your real numbers).
- Auto-selects Plotly charts from result shape; deployed with Docker Compose.